

Sybilant Manual

design and implementation

Generated 2026-08-31

Contents

Introduction 3

Introduction

Sybilant is a Lisp that draws from Clojure and Scheme, extended to let functional, high-level code and raw assembly instructions live side by side in the same source, under a type system that spans both – static where you want the checks, dynamic where you want the freedom.

Sybilant has REPL semantics: the top-level form is the unit of compilation. Sybilant reads, parses, analyzes, compiles, and executes each form before moving on to the next, so every form's effects are observable to the forms that follow it. A file compiles the same way a REPL session runs, as a transcript read top to bottom, one form at a time.

Alongside that interactive mode, Sybilant compiles ahead of time: it emits assembly code files instead of executing forms directly, which you hand to an assembler to produce the final program. Typed Assembly Language underpins the type system that checks that generated assembly.

Sybilant is immutable first. High-level code allows only controlled shared mutability, and low-level code stays type checked throughout. Where that's not enough, you can drop to arbitrary assembly instructions and get the full power of the machine – along with the responsibility to uphold Sybilant's immutability conventions yourself.